

Security+ Cloud Infrastructure CI/CD Lab

Student Walkthrough: LocalStack Desktop, Docker Compose, Terraform, GitHub, and CI/CD

Lab Goal

Students will build a simulated AWS-style cloud environment locally with LocalStack, then push the same Terraform code to GitHub where GitHub Actions automatically tests the infrastructure in a CI/CD pipeline.

1. What You Will Build

- LocalStack Desktop account and auth token
- Docker Compose environment with LocalStack and Kali
- Terraform infrastructure: VPC, subnet, security group, 4 web servers, 1 MySQL server, and S3 log bucket
- GitHub repository
- GitHub Actions CI/CD pipeline
- Security scanning with Checkov

2. Big Picture

Your Windows PC

- ├─ Docker Desktop
- ├─ LocalStack + Kali containers
- ├─ LocalStack Desktop GUI
- ├─ Terraform local testing
- └─ Git pushes code to GitHub

GitHub Actions Runner

- ├─ Temporary Ubuntu machine
- ├─ Starts its own LocalStack container
- ├─ Runs Terraform
- ├─ Runs security scan
- └─ Shows green/red pipeline result

Important

GitHub Actions does not connect to your Windows LocalStack. GitHub creates a separate temporary LocalStack environment inside the GitHub runner.

3. Required Software

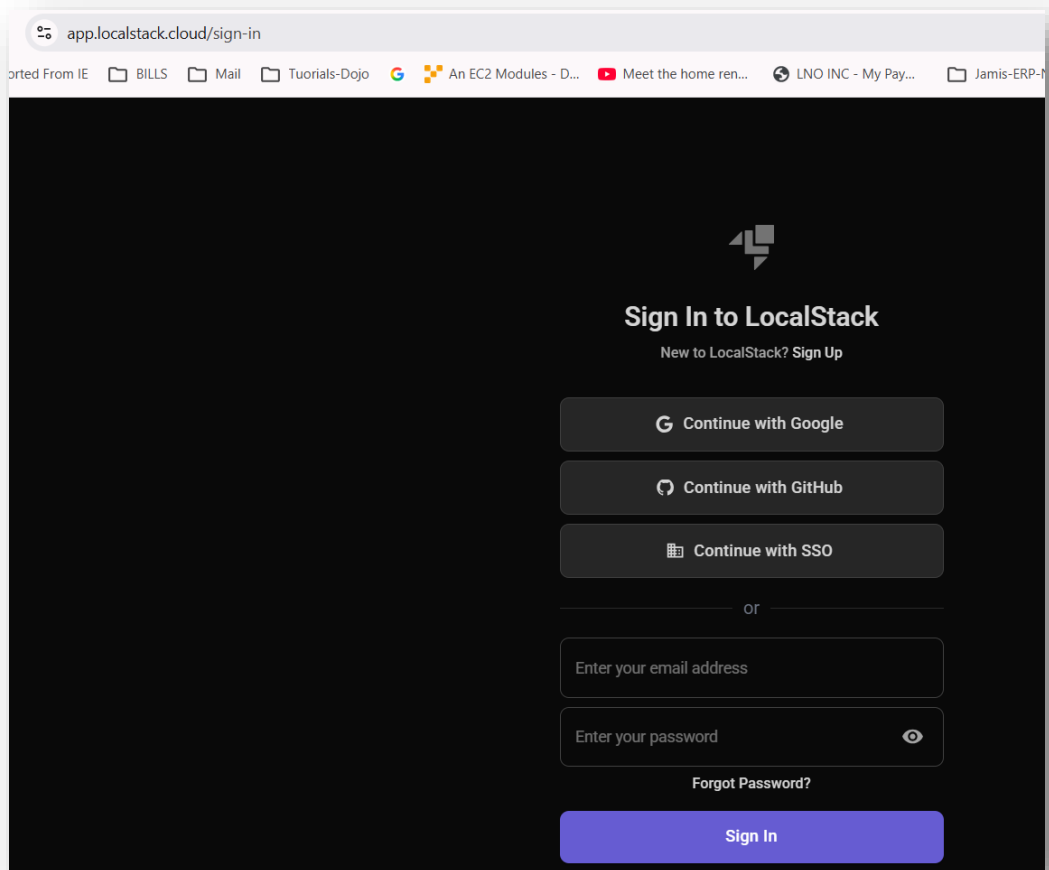
- Docker Desktop for Windows
- Git for Windows
- Terraform
- AWS CLI
- LocalStack Desktop
- GitHub account
- Visual Studio Code or Notepad

4. Step 1 — Create a LocalStack Account

1. Open a browser and go to the official LocalStack website.
2. Click Sign Up, Start for Free, or Get Started.
3. **Create a free account using email, Google, or GitHub sign-in if available.**
4. Verify your email if prompted.
5. Sign in to the LocalStack web dashboard.

Checkpoint: You should be signed in to the LocalStack dashboard.

<https://app.localstack.cloud/>





Set up your Hobby account

Hobby plan

Trial plan

AWS 

Use LocalStack for AWS free for non-commercial projects. No credit card required.

☒

SNOWFLAKE 

Snowflake option is only available for a trial plan.

☐

Jason

Washington

☐ Receive occasional product updates.

Enter LocalStack

By Entering LocalStack, you agree that your use of LocalStack's products and services is governed by our [Terms of Service](#). We will

Jason Washington ▾

 Settings

 Theme  

 Sign Out

 + New Instance

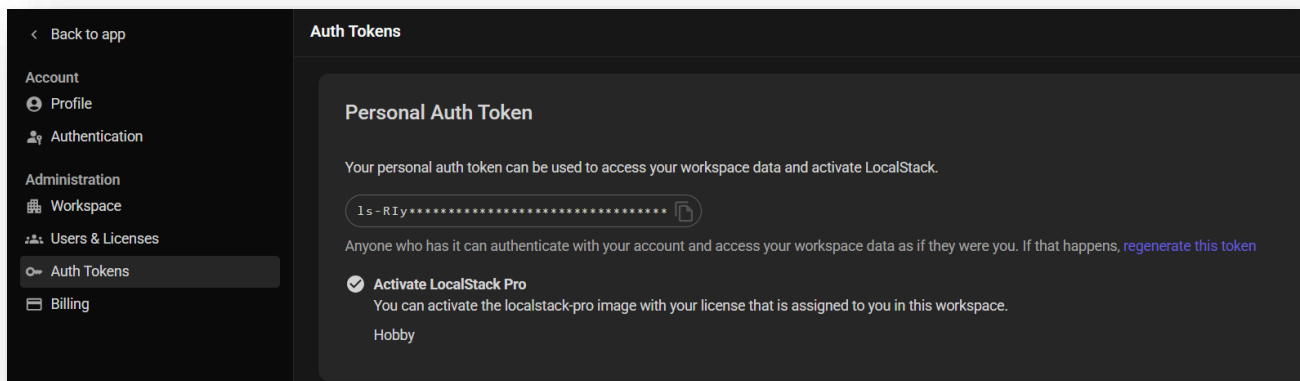
Workspace



Getting Started

Run LocalStack locally with the CLI or Docker

[Install CLI](#) · [View installation methods and docs](#)



5. Step 2 — Install LocalStack Desktop

6. While signed in to LocalStack, find LocalStack Desktop or Desktop App.
7. Download the Windows installer.
8. Run the installer.
9. Open LocalStack Desktop from the Windows Start Menu.
10. Sign in using the same LocalStack account.
11. Confirm Docker Desktop is also running.

If You Cannot Find It

Search the LocalStack website for LocalStack Desktop download. The website layout may change, but the goal is to install the Windows LocalStack Desktop app.

6. Step 3 — Copy Your LocalStack Auth Token

12. Open LocalStack Desktop or the LocalStack web dashboard.
13. Go to Account, Settings, Profile, or Auth Token.
14. Find Auth Token, LOCALSTACK_AUTH_TOKEN, or LocalStack Auth Token.
15. Click Copy Token.
16. Do not share this token.
17. Do not paste it into GitHub, screenshots, email, or chat.

Security+ Tie-In

The auth token is a secret. This demonstrates credential handling, secret protection, and secure configuration management.

7. Step 4 — Create the Project Folder

```
mkdir C:\cloudlab
cd C:\cloudlab
mkdir terraform
```

```
mkdir .github
mkdir .github\workflows
```

8. Step 5 — Create the .env File for the LocalStack Token

```
notepad C:\cloudlab\.env
```

Paste this line. Replace the example text with your real token:

```
LOCALSTACK_AUTH_TOKEN=replace-this-with-your-real-token
```

Important

There should be no spaces around the equals sign.

9. Step 6 — Create .gitignore

```
notepad C:\cloudlab\.gitignore
```

```
# Terraform
.terraform/
terraform/.terraform/
*.tfstate
*.tfstate.backup
.terraform.lock.hcl
```

```
# LocalStack
localstack-data/
```

```
# Secrets
.env
```

```
# Logs
*.log
```

```
# Windows
Thumbs.db
Desktop.ini
```

```
# VS Code
.vscode/
```

10. Step 7 — Create docker-compose.yml

```
notepad C:\cloudlab\docker-compose.yml
```

```
services:
  localstack:
    image: localstack/localstack-pro:3.8.1
    container_name: localstack
    ports:
      - "4566:4566"
      - "4510-4559:4510-4559"
    environment:
      - LOCALSTACK_AUTH_TOKEN=${LOCALSTACK_AUTH_TOKEN}
```

```

- SERVICES=ec2,s3,iam,sts
- DEBUG=1
- PERSISTENCE=1
volumes:
- "./localstack-data:/var/lib/localstack"
- "/var/run/docker.sock:/var/run/docker.sock"
networks:
- lan1

kali:
  image: kasmweb/kali-rolling-desktop:1.18.0
  container_name: kali
  privileged: true
  user: root
  environment:
    VNC_PW: password
  ports:
    - "6901:6901"
  cap_add:
    - NET_RAW
    - NET_ADMIN
  volumes:
    - C:/docker-share:/shared
  tty: true
  stdin_open: true
  networks:
    - lan1

networks:
  lan1:
    driver: bridge

```

Why .env Matters

Docker Compose reads the .env file automatically. This keeps the token out of docker-compose.yml and out of GitHub.

11. Step 8 — Start LocalStack and Kali

```

cd C:\cloudlab
docker compose up -d
docker ps

```

You should see containers named localstack and kali.

12. Step 9 — Verify LocalStack Locally

Note: awscli will need to be installed and the default profile in place

```

curl http://localhost:4566/_localstack/health
aws --endpoint-url=http://localhost:4566 sts get-caller-identity
aws --endpoint-url=http://localhost:4566 s3 ls

```

It is okay if S3 is empty. Terraform has not created the bucket yet.

13. Step 10 — Open LocalStack Desktop

18. Open LocalStack Desktop.
19. Confirm you are signed in.
20. Look for the running local LocalStack instance.
21. Open the instance.
22. If no resources are visible yet, continue to Terraform. Resources appear after Terraform creates them.

Verification Source of Truth

LocalStack Desktop is useful, but AWS CLI and Terraform state are the best verification methods for this lab.

14. Step 11 — Create Terraform main.tf

```
notepad C:\cloudlab\terraform\main.tf

terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

variable "localstack_endpoint" {
  description = "LocalStack endpoint URL"
  type        = string
  default     = "http://localhost:4566"
}

provider "aws" {
  access_key = "test"
  secret_key = "test"
  region     = "us-east-1"

  skip_region_validation      = true
  skip_credentials_validation = true
  skip_metadata_api_check    = true
  skip_requesting_account_id = true
  s3_use_path_style          = true

  endpoints {
    ec2 = var.localstack_endpoint
    s3   = var.localstack_endpoint
    iam  = var.localstack_endpoint
    sts  = var.localstack_endpoint
  }
}

resource "aws_vpc" "securityplus_vpc" {
  cidr_block = "10.10.0.0/16"
  tags = { Name = "securityplus-vpc", Environment = "SecurityPlus-Lab" }
}
```

```

resource "aws_subnet" "public_subnet" {
  vpc_id      = aws_vpc.securityplus_vpc.id
  cidr_block  = "10.10.1.0/24"
  tags = { Name = "securityplus-public-subnet", Environment = "SecurityPlus-Lab" }
}

resource "aws_security_group" "web_sg" {
  name          = "securityplus-web-security-group"
  description   = "Allow web, SSH, and MySQL lab traffic"
  vpc_id        = aws_vpc.securityplus_vpc.id

  ingress {
    description = "Allow HTTP from anywhere for demo purposes"
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    description = "Allow HTTPS from anywhere for demo purposes"
    from_port   = 443
    to_port     = 443
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    description = "Allow SSH from anywhere for demo purposes"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    description = "Allow MySQL only inside the VPC"
    from_port   = 3306
    to_port     = 3306
    protocol    = "tcp"
    cidr_blocks = ["10.10.0.0/16"]
  }

  egress {
    description = "Allow all outbound traffic"
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

resource "aws_instance" "webserver1" {
  ami          = "ami-12345678"
  instance_type = "t2.micro"
  subnet_id    = aws_subnet.public_subnet.id
  vpc_security_group_ids = [aws_security_group.web_sg.id]
}

```

```

    tags = { Name = "webserver1", Role = "Web Server" }
}

resource "aws_instance" "webserver2" {
    ami = "ami-12345678"
    instance_type = "t2.micro"
    subnet_id = aws_subnet.public_subnet.id
    vpc_security_group_ids = [aws_security_group.web_sg.id]
    tags = { Name = "webserver2", Role = "Web Server" }
}

resource "aws_instance" "webserver3" {
    ami = "ami-12345678"
    instance_type = "t2.micro"
    subnet_id = aws_subnet.public_subnet.id
    vpc_security_group_ids = [aws_security_group.web_sg.id]
    tags = { Name = "webserver3", Role = "Web Server" }
}

resource "aws_instance" "webserver4" {
    ami = "ami-12345678"
    instance_type = "t2.micro"
    subnet_id = aws_subnet.public_subnet.id
    vpc_security_group_ids = [aws_security_group.web_sg.id]
    tags = { Name = "webserver4", Role = "Web Server" }
}

resource "aws_instance" "mysql_database" {
    ami = "ami-12345678"
    instance_type = "t2.micro"
    subnet_id = aws_subnet.public_subnet.id
    vpc_security_group_ids = [aws_security_group.web_sg.id]
    tags = { Name = "mysql-database", Role = "Database Server" }
}

resource "aws_s3_bucket" "log_bucket" {
    bucket = "securityplus-log-storage"
    tags = { Name = "securityplus-log-storage", Purpose = "Log Storage" }
}

output "vpc_id" { value = aws_vpc.securityplus_vpc.id }
output "log_bucket_name" { value = aws_s3_bucket.log_bucket.bucket }

```

15. Step 12 — Run Terraform Locally

```

cd C:\cloudlab\terraform
terraform init
terraform validate
terraform apply

```

Type yes when prompted.

16. Step 13 — Verify the Local Terraform Deployment

```

terraform state list
aws --endpoint-url=http://localhost:4566 ec2 describe-vpcs

```

```
aws --endpoint-url=http://localhost:4566 ec2 describe-instances
aws --endpoint-url=http://localhost:4566 s3 ls
```

17. Step 14 — Create a GitHub Repository

23. Go to GitHub.
24. Create a new repository named Security-Cloud-Lab.
25. Do not add a README from GitHub if your local folder already has files.
26. Copy the HTTPS repository URL.

18. Step 15 — Initialize Git and Push to GitHub

Note: If this lab has been ran already use the following commands:
(git remote -v)
(git remote remove origin)

```
cd C:\cloudlab
git init
git add .
git commit -m "Initial Security+ cloud lab"
git branch -M main
git remote add origin https://github.com/washingj77/Security-Cloud-Lab.git
git push -u origin main
```

Reminder

The .env file should not upload because .gitignore protects it.

19. Step 16 — Create the GitHub Actions Pipeline

```
notepad C:\cloudlab\.github\workflows\deploy.yml

name: SecurityPlus LocalStack CI/CD

on:
  push:
    branches:
      - main

env:
  FORCE_JAVASCRIPT_ACTIONS_TO_NODE24: true

jobs:
  terraform-localstack:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
```

- name: Setup Terraform
uses: hashicorp/setup-terraform@v3
- name: Start Free LocalStack with Docker
run: |


```
docker run -d \
  --name localstack \
  -p 4566:4566 \
  -e SERVICES=ec2,s3,iam,sts \
  -e DEBUG=1 \
  localstack/localstack:3.8.1
```
- name: Wait for LocalStack
run: |


```
for i in {1..40}; do
  if curl -s http://localhost:4566/_localstack/health; then
    echo "LocalStack is ready"
    exit 0
  fi
  echo "Waiting for LocalStack..."
  sleep 3
done
echo "LocalStack failed to start"
docker logs localstack
exit 1
```
- name: Terraform Init
working-directory: terraform
run: terraform init
- name: Terraform Validate
working-directory: terraform
run: terraform validate
- name: Terraform Security Scan with Checkov
uses: bridgecrewio/checkov-action@master
with:


```
directory: terraform
framework: terraform
soft_fail: true
```
- name: Terraform Apply
working-directory: terraform
run: terraform apply -auto-approve
- name: Verify LocalStack Resources
run: |


```
aws --endpoint-url=http://localhost:4566 ec2 describe-vpcs
aws --endpoint-url=http://localhost:4566 ec2 describe-instances
aws --endpoint-url=http://localhost:4566 s3 ls
```

Why GitHub Uses Free LocalStack

The local Windows lab uses the student's LocalStack token. The GitHub runner uses a separate free LocalStack container to keep the CI/CD lab simple and avoid putting tokens into GitHub.

20. Step 17 — Push the Pipeline

```
cd C:\cloudlab
git add .github\workflows\deploy.yml
git commit -m "Add LocalStack GitHub Actions pipeline"
git push
```

21. Step 18 — Watch the Pipeline

Go to GitHub > Repository > Actions > SecurityPlus LocalStack CI/CD.

Green means the pipeline successfully started LocalStack, ran Terraform, scanned the code, applied the infrastructure, and verified resources.

Red means one of the steps failed. Open the failed step and read the error.

22. Troubleshooting

LocalStack asks for a license locally

Confirm `.env` exists in `C:\cloudlab` and contains `LOCALSTACK_AUTH_TOKEN=your-token`.

Docker Compose says token is blank

Make sure `.env` is in the same folder as `docker-compose.yml`.

GitHub asks for a LocalStack license

Confirm `deploy.yml` uses `localstack/localstack:3.8.1`, not `localstack/localstack-pro`.

GitHub rejects a large Terraform file

Make sure `.terraform/` and `terraform/.terraform/` are in `.gitignore` before committing.

Everything says up to date

Run `git status` and confirm you are inside `C:\cloudlab`.

LocalStack Desktop shows no resources

Run Terraform apply first, then refresh LocalStack Desktop. Also verify with AWS CLI.

23. Student Reflection Questions

27. Why is the LocalStack token stored in `.env` instead of `docker-compose.yml`?
28. Why does GitHub Actions use its own LocalStack instead of your Windows LocalStack?
29. What does a green pipeline prove?
30. What does Checkov add to the pipeline?
31. Why should Terraform state files not be uploaded to GitHub?